



RÉFÉRENCE & RECUEIL DE REQUÊTES

Manuel de requêtes JQL

Jira — recherche avancée (Jira Query Language)

Tous les exemples s'appuient sur le fil rouge MCO-AVION (projet de clé MCO).

Manuel d'accompagnement de la formation Jira (AGI-JIRA).

Comprendre le JQL

Le JQL (Jira Query Language) est le langage de recherche avancée de Jira. Là où la recherche de base s'appuie sur des menus déroulants (facettes), le JQL permet d'exprimer des critères plus fins et de les combiner librement. Jira sait traduire l'une en l'autre : le meilleur moyen d'apprendre le JQL est d'observer la requête produite par une recherche à facettes, puis de la modifier.

La structure d'une requête

Une requête est une suite de clauses **champ** · **opérateur** · **valeur**, reliées par des mots-clés (AND, OR...) et éventuellement suivies d'un tri (ORDER BY).

```
// projet = MCO, statut = En investigation, trié par priorité décroissante  
project = MCO AND status = "En investigation" ORDER BY priority DESC
```



Les éléments d'une requête JQL : champ, opérateur, valeur, puis tri facultatif.

Note terminologie (Jira Cloud)

Atlassian fait évoluer son vocabulaire : **issue** devient « work item » et **project** devient « space ». Les requêtes JQL existantes continuent de fonctionner ; en cas de doute, l'ancien terme reste valable.

Les champs

Un champ désigne le type d'information recherché. Voici les champs les plus utilisés (la liste complète dépend de votre configuration et de vos champs personnalisés).

Champ	Description	Exemple
project	Le projet (clé ou nom)	<code>project = MCO</code>
type	Type de demande (issuetype)	<code>type = Bug</code>
status	Statut courant	<code>status = "En cours"</code>
statusCategory	Catégorie de statut	<code>statusCategory = Done</code>
resolution	Résolution	<code>resolution = Unresolved</code>
priority	Priorité (ordre implicite)	<code>priority >= High</code>
assignee	Personne assignée	<code>assignee = currentUser()</code>
reporter	Rapporteur	<code>reporter = currentUser()</code>
creator	Créateur de la demande	<code>creator = currentUser()</code>
created	Date de création	<code>created >= -7d</code>
updated	Dernière mise à jour	<code>updated >= startOfWeek()</code>
resolved	Date de résolution	<code>resolved >= startOfMonth(-1)</code>
duedate	Date d'échéance	<code>duedate < now()</code>
labels	Étiquettes	<code>labels in (urgent, mco)</code>
component	Composant	<code>component = Avionique</code>
fixVersion	Version corrigée	<code>fixVersion = "v1.0"</code>
affectedVersion	Version affectée	<code>affectedVersion = "v1.0"</code>
sprint	Sprint	<code>sprint in openSprints()</code>
level	Niveau de sécurité	<code>level = "Diffusion restreinte"</code>
parent	Demande parente	<code>parent = MCO-5</code>
key	Clé de la demande	<code>key = MCO-142</code>
summary	Résumé (texte)	<code>summary ~ "palier"</code>
description	Description (texte)	<code>description ~ vibration</code>
comment	Commentaires (texte)	<code>comment ~ "à valider"</code>
text	Recherche plein texte	<code>text ~ "navigabilité"</code>
watcher	Observateur	<code>watcher = currentUser()</code>
filter	Filtre enregistré (réutilisation)	<code>filter = "Mes faits techniques"</code>

Rappel : le champ « Niveau de sécurité » se cherche avec le tag `level` (et non `security`). Les dates s'écrivent au format `yyyy/MM/dd` ou en relatif (`-1w`, `-3d`, `-2h`).

Les opérateurs

L'opérateur définit la relation entre un champ et une (ou plusieurs) valeurs.

Opérateur	Signification	Exemple
=	égal à	<code>status = Terminé</code>
!=	différent de	<code>status != Terminé</code>
> >= < <=	comparaison (dates, nombres, priorité)	<code>priority > Medium</code>
~	contient (champs texte)	<code>summary ~ "palier"</code>
!~	ne contient pas (texte)	<code>summary !~ test</code>
IN	dans une liste de valeurs	<code>status in ("À faire", "En cours")</code>
NOT IN	hors d'une liste	<code>type not in (Bug, Epic)</code>
IS EMPTY	champ vide (ou IS NULL)	<code>assignee is EMPTY</code>
IS NOT EMPTY	champ renseigné	<code>fixVersion is not EMPTY</code>
WAS	a eu la valeur (historique)	<code>status WAS "En cours"</code>
WAS IN / WAS NOT	historique sur une liste	<code>status was in ("En cours", "Corrigé")</code>
CHANGED	a changé (historique)	<code>status CHANGED</code>

À noter : les opérateurs `~` et `!~` ne s'utilisent que sur des champs texte ; les comparaisons `>` `<` `>=` `<=` concernent les dates, les nombres et la priorité (qui possède un ordre implicite).

Les mots-clés

Les mots-clés relient les clauses et ordonnent les résultats.

Mot-clé	Rôle	Exemple
AND	les deux conditions	<code>project = MCO AND assignee = currentUser()</code>
OR	l'une ou l'autre	<code>priority = High OR priority = Highest</code>
NOT	négation	<code>NOT status = Terminé</code>
EMPTY / NULL	valeur absente	<code>duedate is not EMPTY</code>
ORDER BY	tri des résultats	<code>ORDER BY priority DESC, created ASC</code>

Combiner les conditions

```
// mes demandes non résolues sur le projet MCO
project = MCO AND assignee = currentUser() AND resolution = Unresolved

// priorité haute, non encore terminées – noter les parenthèses
(priority = High OR priority = Highest) AND status != Terminé
```

Précédence des opérateurs

AND est évalué avant OR. Pour un « ou » entre deux conditions, encadrez-les de parenthèses, sinon le résultat peut être surprenant. `A AND (B OR C)` n'est pas `A AND B OR C`.

Les fonctions

Les fonctions renvoient des valeurs dynamiques (l'utilisateur courant, une date relative, les sprints actifs...). Elles s'écrivent avec des parenthèses.

Utilisateurs

Fonction	Renvoie	Exemple
<code>currentUser()</code>	l'utilisateur connecté	<code>assignee = currentUser()</code>
<code>membersOf("grp")</code>	les membres d'un groupe	<code>assignee in membersOf("mco-navigabilite")</code>
<code>currentLogin()</code>	l'heure de connexion courante	<code>created > currentLogin()</code>

Dates et heures

Fonction	Renvoie	Exemple
<code>now()</code>	l'instant présent	<code>duedate < now()</code>
<code>startOfDay()</code>	début du jour (offset possible)	<code>updated >= startOfDay()</code>
<code>startOfWeek()</code>	début de semaine	<code>created >= startOfWeek()</code>
<code>startOfMonth(-1)</code>	début du mois dernier	<code>resolved >= startOfMonth(-1)</code>
<code>endOfMonth()</code>	fin du mois courant	<code>duedate <= endOfMonth()</code>

Sprints et versions

Fonction	Renvoie	Exemple
<code>openSprints()</code>	sprints actifs	<code>sprint in openSprints()</code>
<code>closedSprints()</code>	sprints clôturés	<code>sprint in closedSprints()</code>
<code>futureSprints()</code>	sprints à venir	<code>sprint in futureSprints()</code>
<code>unreleasedVersions()</code>	versions non publiées	<code>fixVersion in unreleasedVersions()</code>
<code>earliestUnreleasedVersion(MCO)</code>	prochaine version du projet	<code>fixVersion = earliestUnreleasedVersion(MCO)</code>

Hiérarchie, liens et activité

Fonction	Renvoie	Exemple
<code>linkedIssues(MCO-5)</code>	demandes liées à une demande	<code>issue in linkedIssues(MCO-5)</code>
<code>issueHistory()</code>	demandes récemment consultées	<code>issue in issueHistory()</code>
<code>watchedIssues()</code>	demandes que j'observe	<code>issue in watchedIssues()</code>
<code>votedIssues()</code>	demandes pour lesquelles j'ai voté	<code>issue in votedIssues()</code>

Rechercher dans l'historique

Les opérateurs **WAS** et **CHANGED** interrogent l'historique des demandes : ce qu'une valeur a été, ou le fait qu'elle ait changé. Ils s'affinent avec des prédicats.

Prédicat	Signification	Exemple
AFTER date	après une date	<code>status was "En cours" AFTER "2026/01/01"</code>
BEFORE date	avant une date	<code>status changed BEFORE -30d</code>
ON date	à une date précise	<code>status was Terminé ON "2026/06/01"</code>
DURING (d1,d2)	pendant un intervalle	<code>status was "En cours" DURING ("2026/01/01", "2026/03/31")</code>
BY user	par un utilisateur	<code>status changed BY currentUser()</code>
FROM valeur	depuis une valeur	<code>status changed FROM Terminé</code>
TO valeur	vers une valeur	<code>status changed TO Terminé</code>

Exemples utiles

```
// faits techniques rouverts (repassés de Terminé à En cours)
project = MCO AND status changed FROM Terminé TO "En cours"

// demandes réaffectées depuis le début du mois
project = MCO AND assignee changed AFTER startOfMonth()
```

Recueil de requêtes prêtes à l'emploi

Copiez-collez et adaptez. Toutes sont écrites pour le projet MCO-AVION (clé MCO).

Mon quotidien

```
// mes demandes ouvertes, Les plus prioritaires en tête
assignee = currentUser() AND resolution = Unresolved ORDER BY priority DESC

// ce que j'ai signalé et qui n'est pas clos
reporter = currentUser() AND status != Terminé

// à traiter aujourd'hui
assignee = currentUser() AND dueDate <= endOfDay()
```

Suivi des faits techniques

```
// en cours d'investigation
project = MCO AND status = "En investigation" ORDER BY created DESC

// ouverts sur le composant Avionique
project = MCO AND component = Avionique AND resolution = Unresolved

// à diffusion restreinte
project = MCO AND level = "Diffusion restreinte"

// non assignés à traiter
project = MCO AND assignee is EMPTY AND resolution = Unresolved
```

Pilotage et qualité

```
// créés cette semaine
project = MCO AND created >= startOfWeek()

// résolus le mois dernier
project = MCO AND resolved >= startOfMonth(-1) AND resolved < startOfMonth()

// en retard (échéance dépassée)
project = MCO AND dueDate < now() AND resolution = Unresolved

// incohérence : terminé mais sans résolution
project = MCO AND status = Terminé AND resolution = Unresolved

// priorité haute non traitée
project = MCO AND priority in (High, Highest) AND status != Terminé
```

Historique et agilité

```
// réouvertes au moins une fois
project = MCO AND status changed FROM Terminé

// modifiées ces 3 derniers jours
project = MCO AND updated >= -3d

// restant à faire dans le sprint courant
project = MCO AND sprint in openSprints() AND status != Terminé

// prévues pour la prochaine version
```



```
project = MCO AND fixVersion = earliestUnreleasedVersion(MCO)
```

Bonnes pratiques & pièges

- **Guillemets** obligatoires pour les valeurs de plusieurs mots : `status = "En investigation"`.
- **Préférez l'ID au nom** pour les statuts, versions et niveaux de sécurité susceptibles d'être renommés : un renommage casse les filtres basés sur le nom, pas ceux basés sur l'ID.
- **Niveau de sécurité** : le tag JQL est `level`, pas `security` ; il n'est disponible qu'en recherche avancée.
- **Texte** : `~` (contient) ne s'applique qu'aux champs texte (`summary`, `description`, `comment`, `text`).
- **EMPTY** : pour « non renseigné », utilisez `is EMPTY` (`assignee is EMPTY`) — pas `= ""`.
- **Précédence** : AND avant OR ; parenthésiez les « ou ».
- **Tri** : `ORDER BY` se place en dernier ; plusieurs critères séparés par des virgules (`ORDER BY priority DESC, created ASC`).
- **Réutilisation** : enregistrez les requêtes utiles en filtres partagés, puis appelez-les avec `filter = "..."`.
- **Limite basic ↔ avancé** : certaines requêtes avancées ne se retraduisent pas en recherche de base (ex. celles utilisant `level` ou l'historique).

Aide-mémoire

Catégorie	À retenir
Opérateurs	<code>= != > >= < <= ~ !~ IN "NOT IN" "IS [NOT] EMPTY" WAS CHANGED</code>
Mots-clés	<code>AND OR NOT ORDER BY (ASC / DESC)</code>
Utilisateur	<code>currentUser() membersOf("grp")</code>
Dates	<code>now() startOfWeek() startOfMonth(-1) endOfMonth() -7d / -1w</code>
Agile / versions	<code>openSprints() closedSprints() earliestUnreleasedVersion(MCO)</code>
À ne pas oublier	<code>level = sécurité</code> <code>~ = plein texte</code> <code>guillemets si plusieurs mots</code>

— Fin du manuel —